

Guía de Laboratorio: Catálogo de Tienda de Artículos de Computación con Vite y FakeStoreAPI

1. Objetivos del Laboratorio y Contexto Técnico

En el ecosistema de desarrollo moderno, la eficiencia y la modularidad son pilares fundamentales. El propósito de este laboratorio es construir un catálogo dinámico para una **"Tienda de Artículos de Computación"** consumiendo datos en tiempo real de una API externa. Abandonaremos prácticas heredadas y nos enfocaremos en herramientas de alto rendimiento.

Utilizaremos **React** como librería core, **Vite** como herramienta de construcción (build tool) y Hooks funcionales para la gestión del ciclo de vida y el estado.

Resultados de aprendizaje esperados:

- **Consumo de API con Fetch:** Implementación de peticiones asíncronas hacia servicios REST.
- **Gestión de Estado y Efectos:** Uso profesional de `useState` y `useEffect` (incluyendo manejo de estados de carga).
- **Renderizado de listas dinámicas:** Transformación de arreglos de datos mediante métodos inmutables.
- **Maquetación con Flexbox:** Diseño de interfaces adaptables basadas en el Modelo de Cajas y Distribución Flexible.

2. Inicialización del Proyecto con Vite

Como Arquitecto de Software, mi primera recomendación es evitar herramientas pesadas y obsoletas como *create-react-app*. Utilizaremos **Vite**, que ofrece un *Hot Module Replacement* (HMR) extremadamente veloz.

Ejecute los siguientes comandos en su terminal:

1. **Inicializar el scaffold:**
2. **Configuración de la plantilla:**
 - Seleccione `React`.
 - Seleccione `JavaScript`.
3. **Instalación y despliegue inicial:**

```
npm create vite@latest
```

3. Análisis y Preparación del Source Context (Fundamentos)

Antes de programar, debemos estandarizar nuestro entorno visual y técnico. Basándonos en las mejores prácticas de CSS, aplicaremos un **Reset Global** para garantizar que el modelo de cajas se comporte de manera consistente en todos los navegadores.

Reset CSS Obligatorio

Añada esto al inicio de su archivo `index.css`:

```
/* Reset Global - Fuente: Estilo Consistente en CSS */
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}
```

Tabla de Conceptos Aplicados

Concepto del Source	Aplicación en el Laboratorio
Arrow Functions	Utilizadas para definir componentes funcionales y callbacks en promesas.
Desestructuración	Técnica para extraer propiedades de los objetos de la API directamente en el renderizado.
Modelo de Cajas	Control de <code>padding</code> y <code>border</code> para definir el área total de cada tarjeta.
Diseño Flexible	Uso de <code>display: flex</code> para organizar el catálogo sin depender de <code>float</code> o <code>inline-block</code> .

4. Estructura de Datos y Consumo de API (FakeStoreAPI)

Para nuestra tienda, consumiremos el endpoint de electrónica. Es vital manejar no solo los datos, sino también el estado de la interfaz durante la petición.

Lógica de Implementación:

- **Estado de carga:** Un arquitecto senior siempre considera la experiencia de usuario (UX). Implemente un estado `isLoading` para feedback visual.
- **Efecto de Montaje:** El `useEffect` debe tener un array de dependencias vacío `[]` para asegurar que la petición se realice solo una vez al montar el componente.

```
// Ejemplo de lógica en App.jsx
const [productos, setProductos] = useState([]);
const [loading, setLoading] = useState(true);

useEffect(() => {
  fetch('https://fakestoreapi.com/products/category/electronics')
    .then(res => {
      if (!res.ok) throw new Error("Error de red");
      return res.json();
    })
    .then(data => {
      setProductos(data);
      setLoading(false);
    })
    .catch(err => {
      console.error("Error al obtener datos:", err);
      setLoading(false);
    });
}, []); // Array vacío: solo se ejecuta al cargar el componente
```

Pro-Tip: En producción, valide que los datos recibidos (como el precio) sean números válidos usando `Number.isNaN()` antes de realizar operaciones matemáticas, evitando errores de tipo en la UI.

5. Maquetación del Catálogo con Flexbox

Aplicaremos las reglas de **Diseño Flexible** vistas en clase. A diferencia de los métodos antiguos que usaban márgenes manuales, utilizaremos la propiedad `gap` para una separación limpia y moderna.

Pasos para la estilización del contenedor:

1. **Activar Flexbox:** Use `display: flex` en el contenedor padre.
2. **Flujo Multilínea:** Aplique `flex-wrap: wrap` para que los productos se ajusten según el ancho de pantalla (Responsividad).
3. **Distribución Profesional:** Utilice `justify-content: center` o `space-evenly` para equilibrar los espacios en el eje principal.
4. **Separación Moderna:** Implemente `gap: 20px` para controlar el espacio entre tarjetas de forma centralizada.

Estilo de la Tarjeta:

```
.product-card {  
  width: 250px;  
  border: 1px solid #ccc;  
  border-radius: 10px; /* Bordes redondeados modernos */  
  padding: 15px;  
  background-color: #fff;  
  transition: transform 0.3s ease;  
}
```

6. Renderizado Dinámico y Props

Para transformar los datos en componentes, utilizaremos el método `.map()`. Siguiendo principios de **código limpio**, desestructuraremos el objeto del producto directamente en los parámetros del mapeo.

Checklist de la Product Card (Accesibilidad y Rendimiento):

- [] **Clave Única (key):** Obligatorio en React para optimizar la actualización del DOM (use el `id`).
- [] **Imagen (image):** Debe incluir el atributo `alt` con el título del producto para accesibilidad (lectores de pantalla).
- [] **Título (title):** Renderizado en un elemento de bloque como `<h3>`.
- [] **Precio (price):** Formateado para lectura clara.

Ejemplo de Renderizado con Destructuración:

```
<div className="catalogo-container">
  {productos.map(({ id, title, price, image }) => (
    <div key={id} className="product-card">
      <img src={image} alt={title} style={{ width: "100%" }} />
      <h3>{title}</h3>
      <p>${price}</p>
    </div>
  ))}
</div>
```

Nota: Para proyectos de mayor escala, se recomienda extraer este bloque a un componente independiente `<ProductCard />`.

7. Interactividad y Estilos de Usuario (Hover y Estados)

La interactividad eleva la calidad percibida de la aplicación. Utilizaremos pseudo-clases para dar feedback visual inmediato al usuario.

- **Feedback de Selección:** Implemente `:hover` en las tarjetas para elevarlas ligeramente (usando `transform: translateY(-5px)`) o cambiar su sombra.
- **Tipografía de Consumo:**
 - Use `font-weight: bold` para que el precio resalte sobre el resto del texto.
 - Aplique `text-align: center` dentro de la tarjeta para una composición balanceada.
 - Defina una `font-family: sans-serif` para asegurar legibilidad en artículos técnicos.